

Lesson 1: Introduction to Python

Python Essentials — Code the Dream



Game Plan for Today

Section	Time	What We'll Do
Warm-Up	5 min	Introductions & icebreaker
I Do	10–15 min	Python basics overview
We Do	15–20 min	Build a function together
You Do	5–10 min	Solo practice exercise
Wrap-Up	5 min	Assignment preview & Q&A

Warm-Up (5 min)

Let's Get to Know Each Other!

Go around and share:

- 1. Your name**
- 2. Where you're joining from**
- 3. One thing you're excited to build or learn with Python**

I Do: Why Python?

- One of the most popular languages in the world
- Used in **data science**, **web development**, **automation**, and more
- Clean, readable syntax — designed to be beginner-friendly
- Huge ecosystem of libraries (Pandas, Matplotlib, Selenium...)

This class focuses on **data engineering** — by the end, you'll build a full data pipeline!

I Do: Python vs. Other Languages

```
# Python
name = "Jazmine"
print(f"Hello, {name}!")
```

```
// JavaScript
let name = "Jazmine";
console.log(`Hello, ${name}!`);
```

I Do: Python vs. Other Languages (Cont.)

Key differences:

- No `let` / `const` / `var` — just assign directly
- No semicolons or curly braces
- Indentation defines code blocks (not `{}`)

I Do: Variables & Data Types

```
name = "Jazmine"      # str (string)
age = 28               # int (integer)
height = 5.8           # float (decimal)
is_student = True      # bool (boolean)
```

I Do: Variables & Data Types (Cont.)

Check a variable's type:

```
print(type(age)) # <class 'int'>
```

What do you think `type("42")` returns?

I Do: Type Conversion

Sometimes you need to convert between types:

```
num_str = "42"  
num_int = int(num_str)    # 42 (now an integer)  
num_float = float(num_int) # 42.0 (now a float)  
back_to_str = str(num_int) # "42" (back to string)
```

I Do: Type Conversion (Cont.)

Why does this matter?

```
age = 30  
# This causes a TypeError:  
message = "I am " + age + " years old."
```

```
# Fix it with str() or an f-string:  
message = f"I am {age} years old."
```

I Do: Control Flow

```
age = 20

if age >= 18:
    print("You're an adult!")
elif age >= 13:
    print("You're a teenager.")
else:
    print("You're a child.")
```

I Do: Control Flow (Cont.)

Notice:

- Colon `:` before each indented block
- `elif` instead of `else if`
- 4-space indentation is standard

Quick poll: What prints if `age = 16` ?

I Do: Loops

```
# For loop – iterate over a range
for i in range(3):
    print(i)  # 0, 1, 2

# While loop – repeat while a condition is True
count = 0
while count < 3:
    print(count)
    count += 1
```

Key detail: `range(3)` gives `0, 1, 2` — it stops *before* 3.

I Do: Functions

```
def greet(name):  
    return f"Hello, {name}!"  
  
message = greet("Luis")  
print(message)  # Hello, Luis!
```

Key ideas:

- `def` keyword defines a function
- `return` sends a value back to the caller
- `return` $\neq$ `print` — this is a common early mistake!

I Do: Error Handling

```
try:  
    result = 10 / 0  
except ZeroDivisionError:  
    print("You can't divide by zero!")
```

- `try` block: code that *might* fail
- `except` block: what to do *if* it fails
- Without this, your program crashes!

The assignment will ask you to catch specific errors.

We Do: Let's Build a Calculator Together

Goal: Write a `calc` function that takes two numbers and an operation.

Let's start simple. **What should our function signature look like?**

```
def calc(a, b, operation="multiply"):
```

What operations should we support?

We Do: Building the Calculator

```
def calc(a, b, operation="multiply"):
    if operation == "add":
        return a + b
    elif operation == "subtract":
        return a - b
    elif operation == "multiply":
        return a * b
    elif operation == "divide":
        return a / b
```

We Do: Building the Calculator (Cont.)

Let's test it:

```
print(calc(10, 5, "add"))      # 15  
print(calc(10, 5))            # 50 (default)
```

What happens if we call `calc(10, 0, "divide")`?

We Do: Adding Error Handling

```
def calc(a, b, operation="multiply"):
    try:
        if operation == "add":
            return a + b
        # ... other operations ...
        elif operation == "divide":
            return a / b
    except ZeroDivisionError:
        return "You can't divide by 0!"
```

What about `calc("hello", "world", "multiply")`?

We Do: Handling TypeError

```
def calc(a, b, operation="multiply"):
    try:
        if operation == "add":
            return a + b
        # ... other operations ...
    except ZeroDivisionError:
        return "You can't divide by 0!"
    except TypeError:
        return "You can't multiply those values!"
```

We Do: Handling TypeError (Cont.)

Nice! We just built something very close to Task 3.

What other operations does the assignment ask for?

- `modulo`, `int_divide`, `power`

Can someone suggest the code for one of these?

You Do: Mini Challenge (5 min)

Write a `greet` function that:

1. Takes a `name` parameter
2. Returns `"Hello, {name}!"` using an f-string

Then try this:

Write a `data_type_conversion` function that:

1. Takes a `value` and a `type_name` (like `"int"`, `"float"`, or `"str"`)
2. Returns the converted value

Stuck? Think about: `int()`, `float()`, `str()`

Assignment Preview

This week's assignment has **10 tasks** — here's the structure:

Tasks	Concepts
1–2	Functions, f-strings, <code>return</code>
3	Calculator with error handling
4	Type conversion with error handling
5	Grading system using <code>*args</code>
6	String repetition with <code>for</code> loop
7	Student scores using <code>**kwargs</code>
8–10	String manipulation challenges

Assignment Tips

Use PyTest to check your work as you go:

```
pytest -v -x assignment1-test.py
```

- `-v` shows which tests pass
- `-x` stops at the first failure (so you can fix one at a time)

Assignment Tips (Cont.)

Workflow:

1. Write the function
2. Test it with a `print()` statement
3. Run PyTest
4. Fix and repeat!

Remember: Functions must `return` values, not just `print` them.

Wrap-Up

Today we covered:

- Variables, data types, and type conversion
- Control flow (`if / elif / else`, loops)
- Functions and `return` values
- Error handling with `try / except`

Quick check: On a scale of 1–5 in the chat, how confident do you feel about writing a Python function?

Getting Help This Week

- Post questions in the **#discussion** Slack channel
- Book a **1:1 mentor session** if you get stuck
- **Start early** — the assignment has 10 tasks and they build on each other
- Read the **PyTest output carefully** — it tells you exactly what didn't match

Next week: Data Structures & File Handling (lists, tuples, dicts, and reading files!)

You've Got This! 

Go build something great this week.

See you next session!